LEEN AI · MASTER PLAN

The AI tuition teacher on a 2D canvas

The single reference for what LEEN AI is, how its core engine works, and how it will be built: product loop, engine design and a phase-by-phase roadmap from today to production.

Project	LEEN AI: AI Personal Teacher System (major project, United College of Engineering and Research, Prayagraj)
Version	1.0, 18 October 2026
Based on	*AI Personal Teacher SaaS: Comprehensive Product Plan* (docs/Utility_Docs), the project synopsis, and the codebase as of commit 36fd7f3
Source file	`docs/master-plan/LEEN_AI_Master_Plan.html` (edit this, then regenerate the PDF)

Contents

1	Purpose and how to use this document	
2	Vision	the product plan in brief
3	Decisions log	everything already decided
4	Current system snapshot	what exists today
5	The daily tuition loop	what a student lives every day
6	Core engine design	the AI-powered 2D canvas
7	Roadmap	13 phases, PDF coverage
8	Phase-by-phase detail	goal, backend, frontend, done when
9	Data model and API outline	
10	Risks and spikes	
11	Demo script	definition of done for the core loop
12	Design calls and open items	
13	Working agreement	how each phase gets built

1 Purpose and how to use this document

This document holds everything needed to keep building LEEN AI without re-deciding things: the product idea, every decision made so far, how the core engine works, and what each future phase delivers.

- **Starting a new phase?** Read chapter 8 for that phase, then the matching parts of chapter 6 (engine) and chapter 9 (tables and endpoints).
- **Wondering why something is the way it is?** Chapter 3 (decisions) and chapter 12 (design calls).
- **Explaining the project to someone?** Chapters 2, 5 and the two diagrams in chapters 5 and 6.
- **Plan changed?** Edit the HTML source and regenerate this PDF (command in chapter 13). The repository copies of the roadmap (`docs/ROADMAP.md`) and engine design (`docs/architecture/CANVAS_ENGINE.md`) must stay in step with it.

> **The one-sentence version.** LEEN AI reads a student's syllabus once, plans one hour of study every day, and teaches that hour on an AI-powered 2D canvas like a real tuition teacher: it draws and explains, asks questions, lets the student answer on the same board, checks their work in red pen, and adapts tomorrow's class to what it learned about them.

2 Vision

Summarised from *AI Personal Teacher SaaS: Comprehensive Product Plan*. LEEN AI is meant to be a full-time teacher, not a doubt-solving tool: it teaches, guides, evaluates and improves a student continuously, and it understands each student's strengths and weaknesses.

Problems it solves

Problem	How LEEN AI answers it
Lack of personalisation: every student gets the same content	A student model (mastery, pace, misconceptions, styles that work) shapes every explanation and every plan
Limited teacher availability	The canvas teacher is available any time, for classes and for doubts
Poor performance tracking: marks don't show understanding	Per-topic mastery from every answer, not just test scores
Static content: books and videos don't adapt	Every explanation is generated live, at the student's level, and re-explained a different way when needed
Lack of discipline: no structured routine	A daily one-hour class, planned automatically, with parents able to see attendance

Users

- **Students**: primary users. School students, competitive-exam students (JEE, NEET) and self-learners.
- **Parents**: monitor progress, receive reports and alerts.
- **Admin**: manages the system.
- **Teachers** (added by this project): create classes that students join with a code. Nothing more is planned for now.

Features in the product plan (§7)

§	Feature	In one line
7.1	Student profile and setup	Class, subjects, goals, learning preferences
7.2	Content upload	Books (PDF), notes, images, understood by the AI
7.3	AI teaching system **(core)**	Teaches from scratch, step by step, with examples, re-explaining in different ways
7.4	Doubt solving	Text, voice and image questions, answered instantly
7.5	Visual learning	Diagrams, infographics, visual explanations; the AI picks the best format
7.6	Video and voice teaching	Audio and video explanations; talk to the AI
7.7	Personalised learning plan	Daily plan: topics, practice, revision; updates with performance
7.8	Homework generator	Assignments at the student's level
7.9	Tests and evaluation	Auto-generated, auto-evaluated, with scores, feedback and suggestions
7.10	Performance tracking	Scores, time spent, weak areas, reports and insights
7.11	Parent dashboard	Progress reports, performance analysis, alerts
7.12	Multi-language	Learn in different languages
7.13	Multi-agent system	Separate intelligent components for teaching, planning, testing, analysis
7.14	Continuous improvement	Learns from interactions and improves over time

The product plan also describes a business model (§9: subscriptions, premium and institutional plans). It is **out of scope** for this project (decision of 18 Oct 2026). Its own four-step roadmap (§10) is replaced by the 13-phase roadmap in chapter 7, which covers every §7 feature.

3 Decisions log

These are settled. Don't re-open them without a reason; change this log if they change.

Foundations (12–13 October 2026)

Area	Decision
Delivery	Plan every phase up front, then build **one phase per session**, each fully working (backend + frontend + tests)
Accounts	Students and parents self-register. Admins create teachers. The first admin comes from `python -m app.cli create-admin`
Look and feel	Light theme by default, with a dark toggle
Frontend language	Plain JavaScript, no TypeScript
Email	Used for password reset only; email verification comes in the final phase
AI model access	Claude Code running on the developer's machine, reached through the **Claude Code bridge**. No API key
Narration voice	Neural Indian-English voices (Neerja, Prabhat) through edge-tts, with the browser voice as fallback. A licensed TTS is the production swap

Core engine and re-plan (18 October 2026)

Area	Decision
Core engine	The **AI-powered 2D canvas** is the core engine. Every teaching feature runs through it
What the engine does	"See the student's syllabus, plan their daily topic for one hour like a tuition teacher, teach like a full human teacher, give questions to solve, check them, and do everything"
Build first	**Richer visuals** and a **two-way board** (the student answers on the canvas)
Lesson style	**Interactive class**: checkpoint questions, plus "raise hand" at any time to interrupt with a doubt
Syllabus	A **one-time upload per subject**: the table of contents of any book, or the syllabus ("Physics has these chapters; each chapter has these topics")
Subjects per day	**One class a day on one subject.** The AI rotates subjects; the student can switch
RAG	Kept **early** (Phase 6) as the synopsis promises, on **pgvector** inside Postgres instead of Qdrant
Business model	**Left out** of the build
Pace	No near deadline: every phase is built fully, in order
Token efficiency	Visuals are **computed by a math engine and libraries**; the AI sends only short semantic commands. Token use is measured on every call
Reference	This master plan is the single reference document

Phase 5 redesign (18 October 2026)

Area	Decision
Student input	The student **never writes on the board**; only the AI does. The student talks to it by **voice** (push to talk) or a **text box**
Interruptions	Student input **never cuts the AI off**. The AI finishes its current step, then works the input into the explanation and carries on, like a real teacher
Moved	English voice input from Phase 12 to Phase 5. The image-input spike and doubts by photo to Phase 6, where the syllabus photo needs them
Teachers	**No new teacher features for now.** What Phase 2 built stays (teachers create classes; students join with a code). Teacher-shared syllabus, test assignment and class analytics are left out

These replace "the student answers on the canvas" and "raise hand to interrupt" above.

4 Current system snapshot

What exists on 18 October 2026, after Phase 4. Phases 1–4 are merged into `main`.

Area	What exists
Stack	React 19, Vite 8, Tailwind v4 (plain JS, pnpm). FastAPI on Python 3.12. Postgres 16, Redis 7, Celery worker with beat embedded. Nginx at `localhost:8081`, Mailpit at `:8025`. Docker Compose
Backend layering	`api/v1/endpoints` → `services` (module functions; they own the commit) → `repositories` (flush only). Every response uses the `ok()` envelope `{success, message, data}`. Roles via `require_roles`; CSRF header check; Redis fixed-window rate limits
Database tables	`users`, `refresh_tokens`, `password_reset_tokens` (migration 0001). `student_profiles` (grade, board, subjects[], learning_level, preferred_language), `parent_student_links`, `classrooms`, `class_enrollments` (0002). `llm_calls` (0004). `board_sessions` and `board_events`, one event per turn (0005; it replaced the `whiteboard_doubts` table of 0003)
AI access	The Claude Code bridge (`infrastructure/claude-bridge/claude_bridge.py`) on `127.0.0.1:8765`: runs `claude -p` with no tools, streams NDJSON, at most 2 runs at once, 300 s limit. The API calls it through the LLM gateway (`app/services/llm/gateway.py`), with model and effort per agent and every call's tokens and cost logged in `llm_calls`. Single turn, text only
Whiteboard tutor	Per-subject prompt in `app/services/whiteboard/prompt.py`. Output validated line by line by `ActionStreamParser` against the Board Protocol v2 schemas (`app/schemas/board/`). Streamed over SSE from `POST /student/doubts`; follow-ups and "explain another way" from `POST /student/doubts/{id}/turns` (6 turns per board). Repeated doubts served from a cache. History and replay. Narration via edge-tts (Neerja/Prabhat), cached in Redis for 7 days
Frontend	Feature folders `features/<name>/`. React Query `['auth','me']` as auth state. Axios with single-flight token refresh. `utils/sse.js` stream parser. Canvas engine in `features/canvas/`: BoardPlayer, visual compilers (mathjs, d3-scale, d3-shape, dagre), pages and camera, strokes, mathText, scene, motion, sprites, narrator; a dev gallery at `/dev/canvas`
Tests	pytest against a real `leen_ai_test` database; the AI is mocked by replacing the gateway's `stream_text`. Vitest + Testing Library on the frontend. Bridge tests run on their own
Not yet built	Student input on the board, RAG, embeddings, pgvector, agents beyond the board teacher, quizzes, plans, mastery tracking, image input

5 The daily tuition loop

This is the product. Everything in the roadmap exists to make this loop work and to report on it.

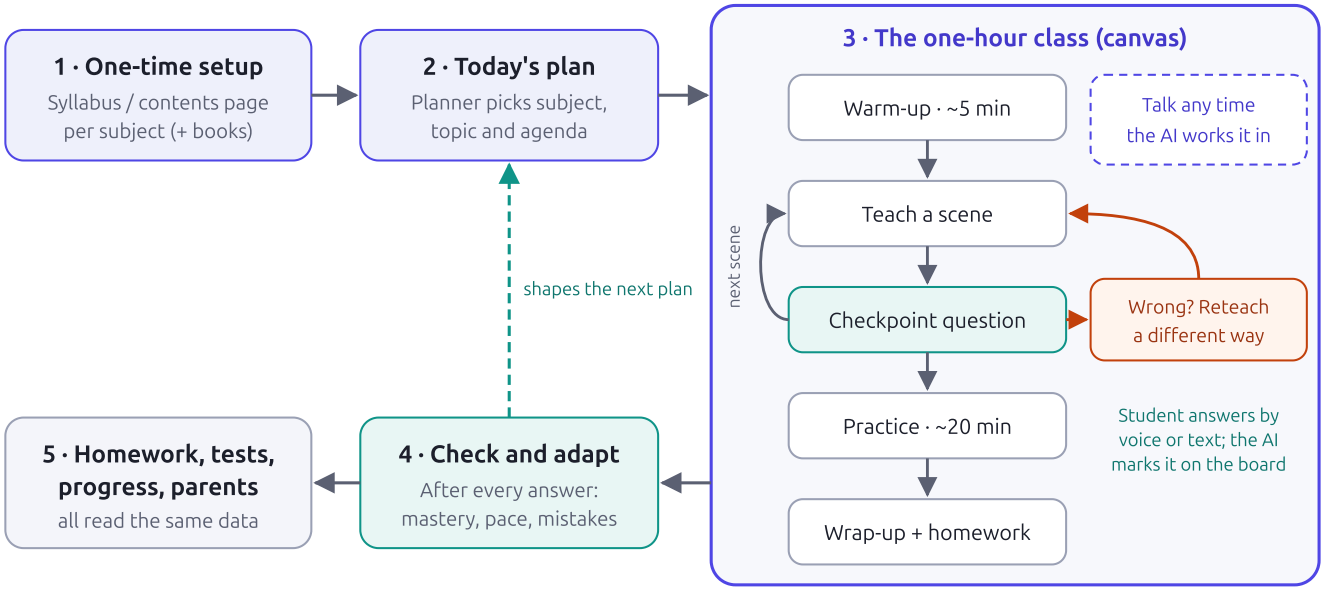


Figure 1. The daily tuition loop. Setup happens once; steps 2 to 4 repeat every day.

1. One-time setup

- Onboarding (already built) asks for class, board, subjects, level and language. It gains: **daily study minutes** (default 60), **study days**, and an optional **target exam and date** (Boards, JEE, NEET).
- **Per subject, the syllabus or a book's contents page is uploaded once**, as a photo or PDF, by the student or parent. The AI extracts *Subject* → *Chapters* → *Topics*. The student reviews it (rename, reorder, delete, add, mark "already done") and confirms. It stays editable.
- Optionally, books and notes are uploaded too. They ground teaching through RAG, with page citations.

2. Today's class

- **One class a day on one subject.** The Planner rotates subjects, favouring weak subjects, near exams, and subjects not studied for a while.
- The plan shows the subject, the topic(s), an agenda with minutes per part, and *why* (new topic, due for revision, unfinished last time).
- The student can switch today's subject or topic.
- It is prepared when the student first opens Today (with a "preparing your class" state), or overnight when the stack is running.

3. The class: one hour on the canvas

Part	Default	What happens
Warm-up	~5 min	2–3 quick questions on revision-due topics of today's subject, including the last class's topic if it was this subject
Teach	~25–30 min	The topic is taught in **scenes** (board pages). Each scene goes idea → visual → worked example, then a **checkpoint** question

Part	Default	What happens
Practice	~20 min	Questions from easy to hard, answered by voice or text. The AI writes each answer on the board and marks it, gives hints on request, and shows the full solution on the canvas when an answer is wrong
Wrap-up	~5 min	A summary board, homework (3–5 questions), and a preview of the next class

- **The split adapts**: a new topic is teach-heavy, a revision day is practice-heavy, a struggling student gets more examples.
- **The student can talk at any time**, by voice or text. The AI never gets cut off: it finishes its current step, answers on the board, and carries on with what was left.
- **A failed checkpoint** makes the AI re-explain *a different way* (another visual or analogy), at most twice; then it moves on and flags the topic as weak.
- **Pause and resume** the same day: the class remembers where it was.

4. Check and adapt

Every answer is graded and updates the **student model**: mastery per topic, misconceptions, pace, and which explanation styles work. The next plan reads it.

5. Homework, tests, progress and parents

- **Homework**: submitted as a photo of paper working or typed, graded in the background, returned with red-pen marks.
- **Tests**: weekly and per chapter, timed and auto-graded; every mistake is explained on the canvas.
- **Progress**: mastery, time spent, weak areas. **Parents**: daily attendance, minutes, topics, scores, alerts, class replays, and a weekly email.

> **Anti-goals.** The design has failed, even if it technically works, if:
>
> - the canvas turns into a slideshow of static text;
> - the student can't speak up at any time, or the tutor loses its place when they do;
> - teaching drifts outside the student's uploaded syllabus;
> - grading says "wrong" without showing *where*.

6 Core engine design

The core engine is the AI-powered 2D canvas and everything behind it. Figure 2 shows its layers.

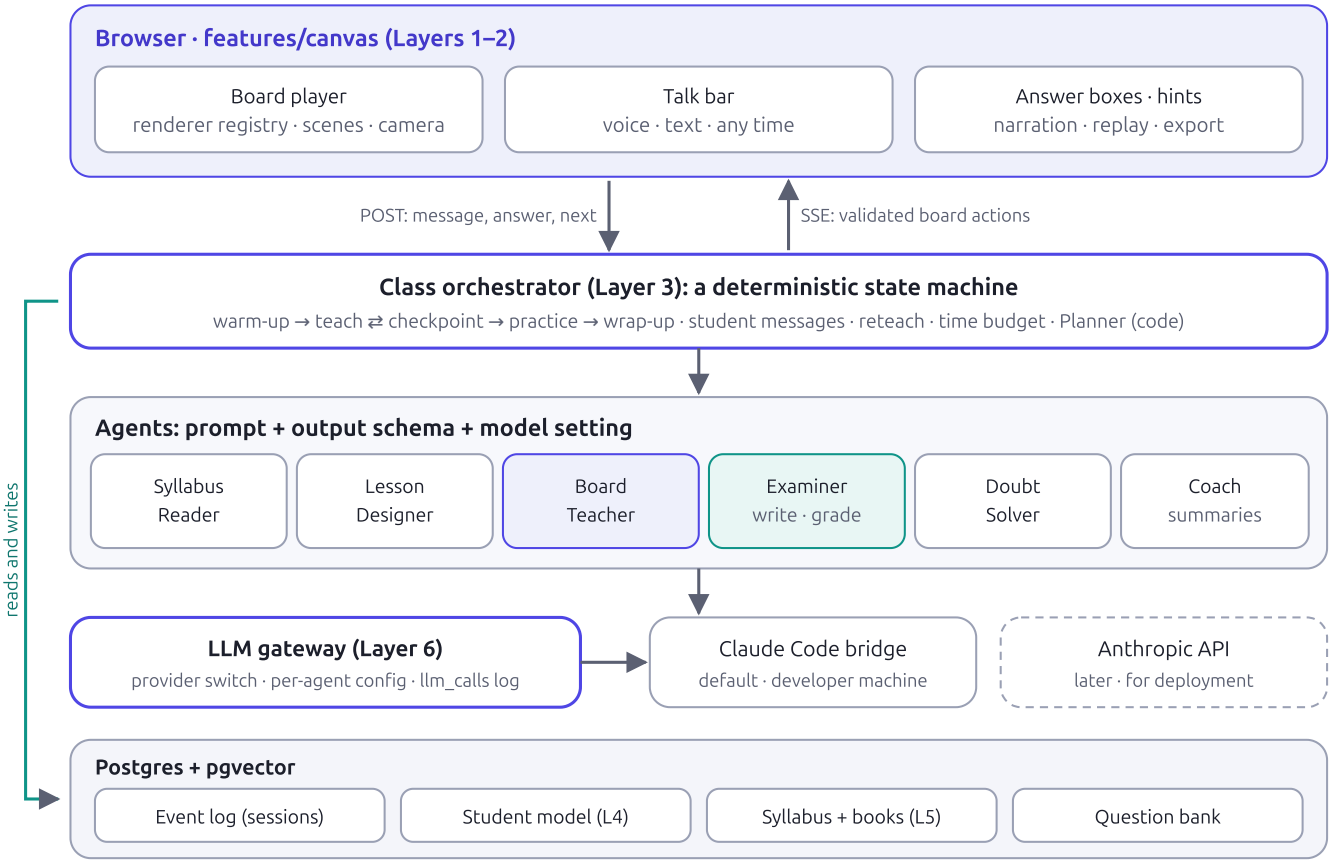


Figure 2. Engine layers. The browser draws and collects answers; the orchestrator decides what happens next; agents write the content; the gateway reaches the model.

Principles

- **The AI says *what*; the engine decides *how it looks*.** The model emits semantic actions such as `plot y = x^2 from -3 to 3` or `triangle A(0,0) B(4,0) C(0,3)`. The engine computes geometry, layout and animation, so the model never computes pixels.
- **Plain data from the model.** Every model output is validated against a pydantic schema before it reaches the browser or the database, as `app/services/whiteboard/parser.py` does today. A prompt-injected input can only change what gets drawn.
- **Deterministic control, AI content.** Code runs the class state machine, scheduling, subject rotation, spaced revision, and grading of choice, number and expression answers. The model writes explanations and questions and judges free-form work. This keeps the loop testable, cheap and predictable.
- **Short streamed steps.** Each AI call covers one scene, one checkpoint or one grading. That fits the bridge's per-call limit, lets the class adapt between steps, and starts drawing within seconds.
- **Token-efficient by design.** If something can be computed, the engine computes it with a math library; the AI only names it. See "Token efficiency" below.
- **Every session is an append-only event log.** Replay, time spent, analytics, the parent view and continuous improvement (§7.14) all read it.

Layer 1 · Board Protocol v2 Built in Phase 4

The shared language between the AI and the canvas: one JSON action per line, streamed. The 15 v1 actions stay unchanged (caption, write, underline, box, erase, line, arrow, rect, circle, clear, pause, object, animate, stop, remove). Every visual also takes `at: [x, y, w, h]` and `color`; without `at` it stacks down the stage on the right (x 920–1540).

Action	Fields the AI sends	What the board computes
`scene`	`title`	Keeps the page as a thumbnail, wipes the board, writes the title
`focus`	`at` (none = zoom back out)	A smooth zoom, at most 2.5×
`column`	`side`: left | right | full	Where writes without x/y flow; each column keeps its place
`plot`	`fns` (≤4, functions of x), `x` [lo, hi], `y`?, `points` {A: [x, y]}, `labels`, `shade` {fn, from, to}, `xlabel`, `ylabel`	Curves (mathjs, split at asymptotes), axes and ticks (d3-scale), shading
`numberline`	`range`, `marks`, `labels`, `interval` {from, to, open}	Ticks, dots, open/closed ends
`geo`	`pts` {A: [x, y]}, `shapes` ("AB", "ABC"), `angles`, `right`, `sides`, `circles`	Fit to the box, angle values, label positions
`table`	`head` + `rows`, or `fn` + `xs`	The values, column widths, text size
`chart`	`kind` bar | pie | line, `labels`, `values`, `title`, `unit`	Scales, bars, slices, axes
`flow`	`nodes`, `edges` [[i, j]] (none = chain), `kind` chart | mind, `dir`	Layout (dagre) or a radial mind map
`diagram`	`kind`: circuit (`parts`, `layout`), lens / mirror (`shape`, `f`, `u`, `h`), fbd (`forces`), atom (`z`, `symbol`), cell (`cell`, `labels`), apparatus (`items`)	The image by the lens or mirror formula, Bohr shells, symbol and label placement
Still to come: `ask`, `mark` and `verdict` (Phase 5, talking to the board); `image` and `cite` (Phase 6, photos and books).		

Everything draws stroke by stroke, like the existing pencil. **Adding a primitive** means adding exactly three things: a pydantic model (backend), a renderer in the frontend registry (`type → render(ctx, action, clock)`), and a short section in the prompt's command reference.

Layer 2 · Talking to the board

The student never writes on the board; only the AI does. The student talks to it at any time, through a **text box** that is always open under the board, or a **push-to-talk mic**. The browser's speech recognition (en-IN) turns speech into text and sends it when they stop speaking; narration is turned down while the mic listens. Browsers without speech recognition show only the text box.

Student input never cuts the AI off. Like a real teacher, the AI finishes the step it is on, then works the message in:

1. The AI writes a board turn in seconds, but playing it takes minutes, so the browser knows how far the board has played when a message arrives.
2. The browser cuts at the end of the current step (the next caption not yet shown). Everything before it plays on.
3. It sends the message with `keep` (how many of the turn's actions stay) straight away, so the AI works while the step finishes. If the AI is still writing the turn, the message waits until it is done.

4. The server records the cut (`cut_at`) and gives the AI short summaries of the board so far and of the part not yet taught, plus the message.
5. The AI acknowledges the message, answers it on the board, then teaches what was left, adapted to it: slower, skipped, or with another picture.

Questions and checking. `ask` (kind choice, number, expression or text) makes the board write the answer line and wait, and the talk bar becomes the answer box. The answer key never reaches the browser. Choice, number and expression answers are checked **in code**, instantly: a small safe evaluator (Python `ast`, no `eval`) reads fractions, roots and π, numbers match within 1%, and expressions match when they agree at several sample values. For number questions the AI sends the sum as `calc`, and code works out the key. The board writes the student's answer in orange with a tick or cross. A wrong answer gets one more try with the hint; after a second, the AI explains the mistake on the board and carries on. Text answers go to the AI, which copies the student's lines onto the board, marks each with `mark` (tick or cross) and records a `verdict`.

Before classes exist (Phase 5), this lives inside **Ask a doubt**: messages mid-explanation, checkpoint questions the AI chooses to ask, and *"Now you try"*, a similar question after the explanation. The daily class reuses all of it.

Layer 3 · Class orchestrator and agents

The orchestrator is a **deterministic state machine**, not an AI deciding what to do next:

```
planned → warmup → teach(scene i) ⇄ checkpoint → practice(question j) → wrapup → completed
            ↘ student message → reply → (carry on)
                                checkpoint failed once or twice → reteach (a different way)
```

A time-budget tracker trims extra practice when the class runs long. Each agent is a prompt, an output schema and a model/effort setting, and all of them call the model through the LLM gateway.

Agent	When it runs	What it produces
Syllabus Reader	A syllabus or contents page is uploaded	A JSON tree of chapters and topics, with order and page numbers when present
Lesson Designer	When today's plan is made	A lesson outline: scenes (goal, chosen visual format, examples), checkpoints, a practice skeleton. Uses the student's books (RAG) when uploaded
Board Teacher	Each scene, reteach and wrap-up	Streamed Protocol v2 actions with captions. Today's whiteboard prompt, evolved
Examiner	Writing questions; grading	Questions with answer key, rubric, difficulty and topic tag; grades for written answers (and photo answers in Phase 9)
Doubt Solver	Student messages; standalone doubts	Board actions, aware of the class and the student's books. Today's doubt flow becomes this agent
Coach	End of class; weekly	Mastery and misconception updates, a student summary and a parent summary

- **The Planner is code, not an agent.** It picks the subject first (rotation weighted by weakness, exam date and days since last studied), then topics, from syllabus order, mastery, revisions due (simplified SM-2 spaced repetition), unfinished topics and daily minutes.
- **Answer-key self-check.** The Examiner solves each generated question a second time, independently. If the answers disagree, the question is discarded.
- **Step streaming.** Each `POST /student/lessons/{id}/advance` carries the student's input (an answer, a message, "next" or "hint") and streams the next step over SSE, ending at an `ask` or at the end of a segment.

Same SSE events, 15-second keep-alive pings and save-on-disconnect as today's doubt stream.

- **Prefetch.** While the student answers a checkpoint, the next scene is generated in the background.

Layer 4 · Student model (memory)

- **Topic mastery** (per student and topic): score 0–1, confidence, attempts, correct answers, last practised, next review date, and a status (not started, learning, practising, mastered). Updated after every graded answer with a simple weighted (Elo-like) rule.
- **Learner profile**: pace, explanation styles that worked, recent misconceptions, language mix. Maintained by the Coach.
- **Student card**: every prompt gets a compact card (under 300 tokens) with class, board, level, language, mastery of the current topic and its prerequisites, top misconceptions, and styles that work.

Layer 5 · Syllabus and knowledge base (RAG)

- **Syllabus.** A contents-page or syllabus photo or PDF goes to the Syllabus Reader; the student confirms the topic tree. It is the backbone for plans, mastery and tests.
- **Books and notes** are ingested in the background (Celery): text extracted from PDFs, scanned pages read through the gateway's vision call, split into chunks by page and heading, embedded with a **free local multilingual embedding model** (no API key; works for Hindi), and stored in **pgvector** (Postgres image `pgvector/pgvector:pg16`).
- **Retrieval** per topic feeds the Lesson Designer and Board Teacher; per doubt, the Doubt Solver. Boards cite pages with `cite` chips. Without books, the AI teaches from its own knowledge, kept to the syllabus topic and the student's board and class.
- **Synopsis alignment.** This is the RAG pipeline the project synopsis promises; pgvector fills the vector-store role the synopsis names as Qdrant.

Layer 6 · LLM gateway

- `app/services/llm/` becomes the single way to call a model: `stream_text(agent, system, prompt, images=[])` and `complete_json(agent, system, prompt, schema, images=[])`.
- The provider is chosen by `LLM_PROVIDER`: `claude_bridge` (default) or `anthropic_api` (for deployment).
- Model and effort are set per agent: the Board Teacher runs fast (sonnet, low effort); the Syllabus Reader and Examiner grading use more effort.
- Every call is logged in `llm_calls`: agent, duration, input, output and cached tokens, cost reported by the bridge, status.
- **Image input** through the bridge is tested first thing in Phase 6 (the syllabus photo needs it), trying in order: (1) `claude -p --input-format stream-json` with image blocks; (2) the bridge saves the image in the run's empty folder and enables *only* Claude Code's Read tool for that run; (3) the Anthropic API for vision calls only, if Satyam agrees to an API key.

Token efficiency

AI tokens are the main running cost and the main source of delay, so the engine spends as few as possible. The rule: **if it can be computed, the engine computes it; the AI only names it.**

Where	How tokens are saved
Graphs	The AI sends `plot y=x^2, x from -3 to 3` (about 25 tokens). The engine evaluates the curve with a math library, instead of the AI writing out hundreds of points
Tables of values	The AI sends the expression and the x values; the engine fills in every cell
Geometry	The AI sends the vertices; the engine computes angles, arcs, lengths and label positions
Diagrams	A template name and a few parameters (`circuit: battery, R1=4Ω, bulb, in series`) instead of 30+ separate line and arrow commands
Flowcharts	Nodes and edges only; a layout library places them
Layout and defaults	The auto-layout cursor means most actions carry no coordinates; colour, size and position are omitted unless they differ from the default
Grading	Choice, number and expression answers are checked in code by a small safe evaluator (1% tolerance, equivalence by sampling), at zero tokens. The AI judges only written answers
Answer keys	For number questions the AI sends the sum (`calc`) and code works out the key, instead of a second AI solve
Student messages	A reply in the middle of an explanation carries two short summaries (the board so far, the part still to teach), not the action lists. The request goes out while the current step is still playing, so the wait is hidden
Prompt size	The command reference is split by subject, so a physics doubt carries only the primitives it needs. Follow-ups get a short board summary (under 200 tokens), not the full action list. The student card stays under 300 tokens. Fixed prompt parts are cached (Claude Code caches automatically; the API provider marks them with `cache_control`)
Model routing	A small, fast model (Haiku) for summaries, syllabus tidy-up and classification; Sonnet at low effort for the board; more effort only for grading free-form work
Reuse	The same doubt (same normalised question, class, board, level and language) is served from cache at zero tokens. Template diagrams cost nothing to redraw
Measurement	Every call logs input, output and cached tokens in `llm_calls`, read from the bridge's result event. Phase 4 records a baseline from v1 doubts and must stay at or below it, despite the richer visuals

Math engine and libraries

Job	Library
Expression evaluation: plots and tables of values	mathjs, loaded only on canvas pages
Axes, ticks, scales and chart shapes	d3-scale, d3-shape
Flowchart and mind-map layout	dagre
Geometry: angles, arcs, label placement	Small in-house vector helpers
Answer checking: numeric tolerance, expression equivalence	A small safe evaluator on Python's `ast` (backend); SymPy only if a later phase needs symbolic algebra

All are free and run locally, with no API key. Exact versions are pinned when each phase is built.

Frontend shape

- The engine moves from `features/whiteboard/engine/` into **`features/canvas/`**: player, renderer registry, scenes and camera, the talk bar (voice and text), answer boxes, timeline and replay, narrator.
- Doubts, classes, homework review, test review and the parent's replay view all mount the same `<Canvas>`.

- Reused as is: `BoardPlayer.js`, `strokes.js`, `mathText.js`, `motion.js`, `scene.js`, `sprites.js`, `narrator.js`, `utils/sse.js`, and the streaming fetch with refresh-and-retry in `doubtService.stream()`.

7 Roadmap

Ordered by dependency. Each phase ships working backend and frontend features together.

#	Phase	PDF §	Status	What ships
1	Auth, RBAC, design system, app shell	§6	Done	Cookie JWT with rotating refresh, roles, design system, landing, sign-in/up, password flows, role dashboards, admin user list
2	Profiles & relationships	7.1	Done	Profiles and avatars, student onboarding, parent–child link codes, teacher classes with join codes, admin user management
3	**Canvas engine v1: AI whiteboard doubts**	7.4	Done	Claude Code bridge, live pencil, 2D animation, neural narration, history and replay
4	**Canvas engine v2: richer visuals**	7.5, 7.13	Done	LLM gateway with token logging; Board Protocol v2 computed by a math engine, so the AI sends short commands; renderer registry; scenes, camera, auto-layout; plots, geometry, tables, diagram templates, flowcharts, charts, number line, chemical equations; board sessions and events (doubts migrated); follow-up questions on the same board; "explain another way"
5	**Talk to the board**	7.4, 7.6, 7.9	Next	The student talks to the board by voice or a text box at any time, and the AI works it in without stopping; ask / mark / verdict; choice, number and expression answers checked in code; written answers copied onto the board and marked by the AI; "Now you try"
6	**Syllabus & knowledge base (RAG)**	7.1, 7.2	Planned	Image-input spike; one-time syllabus/contents upload per subject → editable topic tree; books and notes ingestion, OCR, local embeddings, pgvector; cited retrieval in doubts; doubts by photo
7	**Student model & daily planner**	7.7, 7.10, 7.14	Planned	Daily minutes, study days, exam and date; topic mastery and learner profile; student card in every prompt; Planner with subject rotation and spaced revision; Today page; topic practice that moves mastery
8	**Daily tuition class** (core loop)	7.3, 7.8	Planned	Lesson Designer; class orchestrator (warm-up → teach + checkpoints → practice → wrap-up); talk at any time; reteach a different way; pause and resume; prefetch; homework set; class player; Coach summaries; replay
9	**Homework & tests**	7.8, 7.9	Planned	Homework grading with red-pen review; weekly and chapter tests, timed and auto-graded, mistakes explained on the canvas
10	**Progress & analytics**	7.10, 7.14	Planned	Mastery map, time spent, streaks, weak areas, insights; admin platform and AI-usage stats
11	**Parent dashboard & notifications**	7.11	Planned	Per-child class status, scores, weak areas, alerts, class replays; notification centre; weekly email report
12	**Voice, video & Hindi**	7.6, 7.12	Planned	Hindi voice input; Hindi/Hinglish board and Hindi neural voices; class video export; Hindi interface
13	**Production & polish**	—	Planned	Anthropic API provider for deployment, HTTPS, CI, monitoring, PWA, accessibility, end-to-end tests, email verification, licensed TTS

PDF coverage

§	Feature	Phase(s)
7.1	Profile and setup	2, 6, 7
7.2	Content upload	6
7.3	Teaching	8
7.4	Doubts	3, 4, 5
7.5	Visuals	4
7.6	Voice and video	5, 12
7.7	Learning plan	7
7.8	Homework	8, 9
7.9	Tests	5, 9
7.10	Tracking	7, 10
7.11	Parents	11
7.12	Languages	12
7.13	Multi-agent	4–8
7.14	Improvement	7, 10

What changed from the old roadmap

- The canvas is now the core engine that every teaching feature runs through.
- Standalone "streaming chat" is dropped; follow-up questions happen on the board.
- RAG stays early (Phase 6, on pgvector) and grounds the lessons.
- Quizzes became homework and tests, answered by voice, text or photo.
- Phase 5 redesign: the student talks to the board instead of writing on it.
- The study planner became the daily tuition class.
- Phase numbers changed: the old 8 phases are now 13.
- The business model (§9) is out of scope.

8 Phase-by-phase detail

Each phase lists its goal, the backend and frontend work, and the checks that mean it is done.

Phase 4 Canvas engine v2: richer visuals Done

The AI can draw any common school visual cleanly, and a doubt becomes a multi-turn board session.

> **Result.** On the 8 sample doubts the AI now uses about **60% fewer output tokens** (median 593 vs 1,494.5) and half the commands (17.5 vs 33.5), while choosing plots, diagrams, tables, geometry and flowcharts on its own. Identical doubts are served from a cache with no AI call (68 ms in the live check).

BACKEND

- LLM gateway in `app/services/llm/`: provider switch, per-agent model, effort and timeout, `llm_calls` log. The existing `claude_bridge.stream_text` becomes the bridge provider.
- Protocol v2 schemas split out of `whiteboard.py` into a `board` schema package; `ActionStreamParser` reused.
- Migrations 0004 (`llm_calls`) and 0005 (`board_sessions`, `board_events`, with existing doubts moved in; reversible). Doubt URLs stay the same.
- Follow-ups and "explain another way": `POST /student/doubts/{id}/turns` (SSE) with a short board summary; 6 turns per board.
- Identical doubts (same question and profile) are served from earlier boards with no AI call.
- `board-eval` CLI: sample doubts, token report, fails if the median output grows.
- Prompt command reference covers v2, with guidance on choosing a visual format, split by subject so each prompt carries only what it needs.
- Token usage (input, output, cached) read from the bridge's result event into `llm_calls`; a baseline measured from v1 doubts.

FRONTEND

- Engine extracted to `features/canvas/`.
- Renderer registry, scenes with a thumbnail strip, camera `focus`, auto-layout cursor.
- Renderers computed by the math engine: mathjs for plots and tables of values (never `eval`), d3-scale/d3-shape for axes and charts, dagre for flowcharts, in-house geometry helpers; plus diagram templates and the number line. (The `image` action moved to Phase 6, with photo doubts.)
- Doubts page: follow-up box and "Explain another way" button.

DONE WHEN

- A developer-only gallery renders every primitive.
- Sample doubts in maths, physics, chemistry and biology each pick and draw a fitting visual.
- Unit tests cover the expression parser, layout and schema validation.
- Old doubts replay unchanged.
- Median output tokens per doubt stay at or below the v1 baseline.

Phase 5 Talk to the board Next

The student talks to the board by voice or text at any time. The AI finishes its step, works what they said into the explanation like a real teacher, asks questions and checks the answers.

BACKEND

- Board actions `ask` (choice, number, expression or text; the answer key is stripped before it reaches the browser), `mark` (a tick or cross on a written line) and `verdict`.
- Mid-explanation replies: a turn carries the message and `keep`; the earlier turn records `cut_at`; the prompt carries summaries of the board so far and of the part still to teach.
- "Check" turns (explain a wrong answer) and "Now you try" turns (a practice question); 12 turns per board.
- `POST /student/doubts/{id}/answers`: choice, number and expression answers checked in code by a safe evaluator, with two tries; answers stored as board events.
- A questions module in the board prompt.

FRONTEND

- Talk bar under the board: a text box; a push-to-talk mic (browser speech recognition, en-IN, narration turned down while listening); answer boxes for each kind; hint; "Show me"; "Explain another way"; "Now you try".
- Player: cut at the end of the current step, wait at an `ask`, write the student's answer with its tick or cross.
- Replays show the student's messages and answers.

DONE WHEN

- A message sent mid-explanation is answered after the current step, and the AI carries on with what was left instead of starting over.
- Choice, number and expression answers are marked instantly with no AI call; a written answer is copied onto the board with the wrong line crossed in red.
- "Now you try" sets a question and checks the answer.
- Tests cover the evaluator and grading, the cut, the prompts and the player's question flow.

Phase 6 Syllabus & knowledge base (RAG)

The AI knows each student's syllabus and can teach from their own books.

BACKEND

- Syllabus upload (photo or PDF) → Celery → Syllabus Reader (vision) → draft tree → student confirms.
- `syllabus_*` tables, one tree per student and subject.
- Books and notes: `documents` upload with size limits; Celery extracts PDF text, reads scanned pages by vision, splits into ~500-token chunks with page references, embeds with a small local multilingual model (chosen at build time), stores in pgvector with an HNSW index.
- `retrieve(student, query, subject/topic)` searches only the student's own documents. Doubts use it, with `cite`.

FRONTEND

- Syllabus page: upload, review the tree (edit, reorder, mark done), confirm.
- Documents list with ingestion status.
- `cite` chip that opens the page.

DONE WHEN

- A contents-page photo becomes a correct tree.
- A doubt cites the right textbook page.
- Tests cover the chunker, and retrieval with fake embeddings.

Phase 7 Student model & daily planner

The AI knows what the student has mastered and plans each day's class.

BACKEND

- Onboarding fields: daily minutes, study days, exam and date.
- `topic_mastery` updated from graded answers (now tagged with a topic); `learner_profiles`; student-card builder.
- Planner as pure, unit-tested functions: subject rotation score, topic choice, spaced revision, agenda minutes.
- `daily_plans`; `GET /student/today` (makes the plan if missing); `POST /student/today/switch`; overnight pre-generation by Celery beat at 02:00 IST.
- `question_items` (moved from Phase 5), tagged with topics.
- "Practice this topic": a short talk-to-the-board session that moves mastery, so the phase is useful before classes exist.

FRONTEND

- New onboarding step.
- Today page: agenda, why this topic, switch, practice.
- Mastery chips on the Syllabus page.

DONE WHEN

- The Planner picks the expected subject and topic on seeded data (unit tests).
- Practice moves mastery.
- Today shows the plan.

Phase 8 Daily tuition class (the core loop)

The full one-hour interactive class works end to end.

BACKEND

- Lesson Designer → `lesson_outlines`, made together with the plan.
- Class orchestrator: state stored on the board session; steps warm-up, teach, checkpoint, reteach, practice, wrap-up, doubt; time budget; next scene prefetched and cached in Redis.
- Endpoints: start or resume, `advance` (SSE), pause, replay.
- Student messages work as in Phase 5: the AI finishes its step, answers, and carries on with the lesson.
- Wrap-up creates a homework row in `assignments`.
- Coach summary: mastery and profile updates, plus the summary parents will see.

FRONTEND

- Class player at `/student/lesson/:id`: agenda progress, timer, talk bar, answer boxes, scene strip, end-of-class summary.

DONE WHEN

- The demo script (chapter 11) passes end to end on the live bridge.
- The state machine is unit-tested with fake agents.
- A class resumes after a page reload.

Phase 9 Homework & tests

Homework and tests are checked by the AI, and every mistake is explained.

BACKEND

- `submissions`, `submission_grades`; background grading in Celery.
- Weekly tests created automatically each Sunday from the week's topics; chapter tests on demand; timed attempts with autosave.

FRONTEND

- Homework list; answers typed, spoken or photographed.
- Red-pen review with "show me on the board" per mistake.
- Timed test player and results.

DONE WHEN

- A homework photo comes back marked.
- A test is taken, auto-graded and reviewed.

Phase 10 Progress & analytics

The student and admin can see how learning is going.

BACKEND

- Aggregates from topic mastery, board events (time spent) and grades.
- Weekly Coach insights.
- Admin AI usage from `llm_calls`.

FRONTEND

- Student: mastery map (subject → chapter → topic), time, streak, score trends, weak areas, recommended next topics.
- Admin: platform and AI-usage stats.

DONE WHEN

- The numbers match seeded data in tests.

Phase 11 Parent dashboard & notifications

Parents see each child's progress, get alerts, and receive a weekly report.

BACKEND

- `notifications` table.
- Triggers: two missed classes, a falling score, a test completed, the weekly report ready.
- Weekly email report (Celery beat, Sunday evening IST) through the existing email task.

FRONTEND

- Per-child view: today's class, minutes, topics, scores, weak areas, alerts, read-only class replays.
- Notification bell and centre.

DONE WHEN

- Triggers fire in tests and the weekly email arrives in Mailpit.

Phase 12 Voice, video & Hindi

Students can learn by voice, export a class as a video, and learn in Hindi.

BACKEND

- Hindi voices (`hi-IN-SwaraNeural`, `hi-IN-MadhurNeural`) through edge-tts.
- Hindi and Hinglish board-language modes in the prompts.

FRONTEND

- Hindi voice input (hi-IN), and a local speech model if browser recognition is not enough.
- Hindi interface text.
- Class video export: canvas layers and narration recorded to WebM.

DONE WHEN

- A full class runs in Hindi, a doubt spoken in Hindi works, and an exported video plays.

Phase 13 Production & polish

The product is ready to deploy safely.

- Anthropic API provider implemented and tested.
- HTTPS, secure cookies, Content Security Policy review.
- CI on GitHub Actions: ruff, pytest, eslint, vitest, build.
- Monitoring, backups, installable app (PWA), accessibility audit, code splitting.
- End-to-end tests (Playwright) of the core loop.
- Email verification; licensed TTS; documentation refresh.

9 Data model and API outline

New tables by phase

Phase	Tables and columns
4	`board_sessions` (kind: doubt, class, homework, test review; student; subject; topic; status; protocol version; minutes) and `board_events` (session, sequence number, kind, payload as JSONB). Existing `whiteboard_doubts` rows are migrated in. `llm_calls`
5	No new tables: an `ask` lives in its board turn, answers are `board_events` of kind `answer`, and turns gain `cut_at`
6	`syllabus_uploads`; `syllabus_subjects`, `syllabus_chapters`, `syllabus_topics` (per student); `documents`; `document_chunks` (pgvector embedding)
7	`student_profiles` gains daily_minutes, study_days, target_exam, exam_date. `question_items` (subject, topic, kind, prompt, options, answer key, rubric, difficulty, source; moved from Phase 5). `topic_mastery`; `learner_profiles`; `daily_plans` (student, date, subject, agenda as JSONB, status)
8	`lesson_outlines` (topic, student, outline as JSONB); `assignments` (homework or test; student; items; due date; time limit). The class wrap-up creates homework
9	`submissions`, `submission_grades`
11	`notifications`; weekly report snapshots

All IDs follow the existing convention (36-character UUID strings). Every migration must be reversible.

API outline

Student scope unless noted. All JSON responses use the `{success, message, data}` envelope; streams use SSE.

Area	Endpoints
Syllabus and books	`POST /student/syllabus/uploads` · `GET` and `PUT /student/syllabus/{subject}` · `POST /student/documents`
Today	`GET /student/today` (makes the plan if missing) · `POST /student/today/switch`
Lesson (daily class)	`POST /student/lessons` (start or resume) · `POST /student/lessons/{id}/advance` (SSE) · `GET /student/lessons/{id}` (replay) · `POST /student/lessons/{id}/pause`. Named lessons because `/student/classes` is already the classroom API
Doubts	Existing `/student/doubts` · `POST /student/doubts/{id}/turns` (follow-up, reply mid-explanation, another way, check, "Now you try") · `POST /student/doubts/{id}/answers` · photo doubts in Phase 6
Homework and tests	`GET /student/assignments` · `POST /student/assignments/{id}/submit` · review endpoints
Progress	`GET /student/progress` · `GET /parent/children/{id}/progress`

10 Risks and spikes

#	Risk	Why it matters	Plan
1	**Image input through the bridge**	Reading syllabus photos, photo doubts and marking homework photos all need the model to see images. The bridge is text-only today	A spike opens Phase 6 with a three-step fallback: stream-json image blocks → Read tool limited to the run's folder → Anthropic API for vision only (only with Satyam's agreement)
2	**Latency**	A step takes 5–20 seconds of model time; a class must not feel stalled	Plans prepared ahead (or on first open, with a progress state); drawing starts on the first streamed action; the next scene is prefetched during checkpoints
3	**Bridge capacity**	At most 2 runs at once	Fine for a demo with 1–2 students; raise `--max-concurrent` when needed; the API provider is the path for real users
4	**Board clutter**	AI-placed content can overlap	Auto-layout cursor, columns, scenes and camera focus
5	**Wrong answer keys**	A wrong key marks a right answer wrong and destroys trust	Number keys computed in code from the AI's `calc`, numeric tolerance; the student can say "I think that's wrong" and the AI re-checks
6	**Cost per class**	About 15–30 model calls per hour-long class	Math-engine visuals, subject-split prompts, caching, model routing and grading in code (chapter 6); tokens per agent visible in `llm_calls`
7	**Embedding model size**	A local model adds a few hundred MB to the images	Loaded once per process; ingestion runs only in Celery
8	**Speech recognition**	Browser speech recognition works only in Chrome and Edge, and sends audio to the browser maker's service	The text box always works; a local speech model can replace it in Phase 12

11 Demo script

The definition of done for the core loop, at the end of Phase 8. It is also the script for showing the project to evaluators.

1. The student uploads a photo of their physics book's contents page once, and confirms the topic tree. They upload the textbook PDF too.
2. **Today's class** shows a 60-minute Physics plan for "Motion in a straight line", chosen by subject rotation.
3. In class, the AI draws a velocity–time graph and cites the textbook page.
4. It asks a checkpoint question. The student says the working aloud; the AI writes it on the board, crosses line 2 in red, and re-explains with a different picture.
5. Mid-scene the student types a doubt. The AI finishes its sentence, answers it on the board, and carries on with the lesson.
6. Practice answers are typed or spoken and marked on the board. Homework is set. The mastery bar for that topic moves.

> Later phases extend the demo: the homework photo comes back marked (Phase 9), the progress page shows the mastery change (Phase 10), and the parent sees today's class and the weekly report (Phase 11).

12 Design calls and open items

Design calls

Made while planning; each can be overridden.

- The engine stays on the **Claude Code bridge** behind a provider switch. The image fallback enables only the Read tool; a paid API key is used only with Satyam's agreement.
- A **free local embedding model** feeds pgvector, so no API key is needed.
- **Math engine libraries**: mathjs, d3-scale/d3-shape and dagre in the browser; a small safe evaluator for answer checking on the backend.
- **Today's plan** is made on the first open of the day, and overnight too when the stack is running.
- **Daily minutes default to 60.** Minutes and study days are per student, not per subject.
- The **Planner and class flow are code**; the AI writes the content.
- **Class split**: warm-up 5, teach 25–30, practice 20, wrap-up 5 minutes, adapting per day. At most **two reteaches** per checkpoint.
- **Two ways to talk to the board**: voice and text, never writing on it. A spoken message is sent when the student stops speaking.
- **Two tries** for a code-checked answer; after that the AI explains the mistake on the board.
- **Phase 5 lives inside Ask a doubt**, so it ships something usable before classes exist.
- **Teachers keep what Phase 2 built** (classes with join codes); no further teacher features are planned for now (Satyam, 18 Oct 2026).
- **Standalone chat is dropped** in favour of follow-up questions on the canvas.
- The engine design lives in `docs/architecture/CANVAS_ENGINE.md`, next to `ARCHITECTURE.md`.

Open items

- Result of the image-input spike (start of Phase 6).
- Which local embedding model (Phase 6): small, multilingual, 384-dimensional.
- Chart library for analytics (Phase 10).
- Licensed TTS vendor for production (Phase 13).

13 Working agreement

- **One phase per session.** Branch `feature/phase-N-<slug>` from `main`, or stacked on the previous phase while that one is unmerged.
- **Each phase delivers backend and frontend together, with tests**: pytest with a mocked LLM gateway; vitest for engine maths and layout.
- **Follow the existing patterns**: feature folders on the frontend; endpoints → services → repositories on the backend; the response envelope; `require_roles`; reversible migrations.
- **When a phase ends**:
  - update the status in `docs/ROADMAP.md`;
  - update `CANVAS_ENGINE.md` and `ARCHITECTURE.md` if the design moved;
  - update the "Soon" labels in `frontend/src/config/navigation.js` and the dashboards;
  - regenerate this PDF if the plan changed.
- **Commits** follow Conventional Commits (`feat(backend):`, `feat(frontend):`, `docs:`). Nothing is pushed unless asked.

Regenerating this PDF

Edit `docs/master-plan/LEEN_AI_Master_Plan.html`, then run from the repository root:

```
google-chrome --headless=new --no-pdf-header-footer \
  --print-to-pdf=docs/LEEN_AI_Master_Plan.pdf \
  docs/master-plan/LEEN_AI_Master_Plan.html
```

Related documents

File	What it is
`docs/ROADMAP.md`	The phase table, kept in step with chapter 7
`docs/architecture/CANVAS_ENGINE.md`	The engine design, kept in step with chapter 6
`docs/architecture/ARCHITECTURE.md`	The system as built: topology, auth, endpoints, the whiteboard
`docs/setup/SETUP.md`	Running the stack and the Claude Code bridge
`docs/standards/`	Contributing, coding standards, development guide
`docs/Utility_Docs/`	The original product plan PDF